



Università degli Studi di Camerino

SCHOOL OF SCIENCE AND TECHNOLOGY

Course in Deep Learning - Computer Science (Class LM-18)

Binary Images Classification of metastatic tissue: defining and training a model to classify disease diagnosis

Group members

Tommaso Cosci

Alessio Tartufoli

Contents

1	Introduction and Problem Statement	1
1.1	Project Objectives & Clinical Priority	1
1.2	Technical Challenges & System Constraints	1
2	Dataset Analysis and Handling	3
2.1	Dataset Structure and Provenance	3
2.2	Practical Implementation on Google Colab	4
3	Methodology and Implementation	6
3.1	Experimental Design and Comparative Protocol	6
3.2	Baseline Model: CNN "From-scratch"	7
3.2.1	Data Streaming and Memory Caching Strategy	7
3.2.2	Topological Architecture and Multi-Level Regularization	7
3.2.3	Training Execution and Accelerated Optimization Profile	8
3.3	Deep Residual Learning (ResNet)	9
3.3.1	Architectural Structure: The Residual Functional Mapping	9
3.3.2	Data Management and Local Host Optimization	10
3.3.3	Topological Hierarchy and Optimization Schedule	10
3.4	Hybrid G-CNN: Integrating Equivariance and Depth	11
3.4.1	p4 Symmetry and Coordinate-Free Feature Representations	11
3.4.2	Architectural Layout and Group Coordinate Collapsing	11
3.5	Transfer Learning: DenseNet121 Suite	13
3.5.1	Preprocessing Customization and Data Augmentation Architecture	13
3.5.2	Two-Stage Asymmetric Optimization Scheme	13
3.5.3	Out-of-Core Test Ingestion Pipeline	14
4	Results and Discussion	15
4.1	Training Dynamics: Learning Curves	15
4.2	Global Performance Comparison	16
4.2.1	Discriminative Capacity: ROC and AUC Analysis	16
4.2.2	Architectural and Clinical Discussion	18
4.3	Qualitative Analysis: Confusion Matrices	19
4.4	State of the Art	20
	References	21

List of Figures

2.1	Distribution of working samples across the dataset splits. Each set maintains a strict 50/50 ratio to ensure unbiased learning.	4
2.2	Some examples to compare healthy and diseased tissues.	4
3.1	Topological architecture of the custom CNN From-Scratch baseline. The pipeline highlights the $80 \times 80 \times 3$ spatial resampling, the progressive 3×3 convolutional hierarchy ($32 \rightarrow 64 \rightarrow 128 \rightarrow 256$), and the multi-level regularization block integrating Spatial Dropout and L_2 weight penalty.	8
3.2	Topological scaling pipeline of a standard ResNet layout. Residual identity mappings bypass traditional stacked layers to directly preserve deep gradient propagation highways.	9
3.3	Architectural layout of the Group-Equivariant G-ResNet implemented in <code>GCNN.py</code> . The diagram illustrates the lifting convolution stage (<i>P4LiftingConv2D</i>) projecting input patches onto the discrete $p4$ symmetry group, the equivariant residual backbone, and the custom <i>GroupPool</i> layer executing manifold collapsing ($\max_{g \in p4}$) for rotation-invariant tissue classification.	12
3.4	Structural connectivity matrix inside a baseline Dense Block setup. Each internal processing layer receives unified direct inputs from all historical preceding blocks, maximizing out-of-domain feature reuse configurations.	14
4.1	Learning curves (Loss and Accuracy) across all architectures. The scratch-trained configurations require more epochs to stabilize, while DenseNet121 benefits from pre-trained weights and a conservative learning rate ($1e-5$).	15
4.2	ROC curves and absolute AUC values computed over the test data distribution.	17
4.3	Performance comparison across the evaluated architectural paradigms on the PCam test benchmark.	18
4.4	Confusion Matrix comparison evaluated across the test data samples.	19

List of Tables

4.1	Global performance comparison on the PCam test set under the unified screening protocol ($Threshold = 0.35$). Bold values indicate the top performer for each metric.	16
-----	---	----

1. Introduction and Problem Statement

This project, developed for the Deep Learning course, focuses on the binary classification of metastatic tissue using the **PatchCamelyon (PCam) benchmark**. The objective is to design and compare deep learning pipelines capable of distinguishing between healthy and tumor-positive lymph node patches, supporting computer-aided digital pathology and automated oncological screening.

1.1 Project Objectives & Clinical Priority

The primary objective of this study is to design, implement, and evaluate different Convolutional Neural Network (CNN) architectures for automated histopathological image classification. After defining the task as a binary computer vision problem, our experimental protocol explicitly **prioritizes Recall (Sensitivity)** and the Area Under the ROC Curve (AUC) as the main performance indicators.

In a clinical screening environment, the cost of a False Negative, meaning a missed metastatic region, is significantly more critical than that of a False Positive, which would only require additional manual review. For this reason, the project focuses on a standardized comparative study rather than on optimizing a single model in isolation. We evaluate how different architectural choices, regularization strategies, and data handling pipelines affect the detection of malignant patterns under common experimental conditions and a unified decision threshold.

1.2 Technical Challenges & System Constraints

A central challenge of this task lies in the specific labeling protocol of the PCam dataset. Although each input image has a resolution of 96×96 pixels in the Hematoxylin and Eosin (H&E) color space, the ground-truth binary label is determined exclusively by the central 32×32 region, where the presence of at least one tumor pixel defines a positive sample.

Our implementation addresses this constraint through three main aspects:

- **Topology-Dependent Spatial Contextualization:** Since only the central region determines the label, the preprocessing strategy is adapted to the specific model architecture. Scratch-trained models use a cropped input size of $80 \times 80 \times 3$ to reduce peripheral noise while preserving enough contextual information around the central region. Transfer Learning models instead keep the original $96 \times 96 \times 3$ input size required by the pre-trained backbone.

- **Geometric Inductive Bias:** Histopathological images do not have a preferred orientation, since cellular structures and malignant patterns remain diagnostically relevant regardless of their rotation. For this reason, we investigate the use of data augmentation and group-equivariant convolutions based on the discrete cyclic group $p4$. This allows the models to better handle rotational variability without relying only on increased depth or parameter count.
- **Out-of-Core Data Engineering:** Managing the PCam dataset in a cloud-based environment requires an efficient data pipeline to avoid memory and I/O bottlenecks. To address this, our implementation uses an out-of-core strategy based on HDF5 containers, asynchronous data loading, and background prefetching, allowing the GPU to remain efficiently utilized during training.

2. Dataset Analysis and Handling

This project is based on the **PatchCamelyon (PCam) benchmark**, a large-scale histopathological dataset derived from the Camelyon16 Challenge. The complete benchmark consists of 327,680 color patches extracted from lymph node tissue sections stained with Hematoxylin and Eosin (H&E).

2.1 Dataset Structure and Provenance

To ensure a fair comparison between the evaluated architectures while keeping the computational cost manageable, our experimental pipeline uses a standardized subset extracted from the original data splits:

- **Source Constraints:** The individual patches were originally digitized at $40\times$ magnification and subsequently downsampled to $10\times$ ($0.972\text{ }\mu\text{m}$ per pixel). This resolution provides enough tissue context for the classification task while keeping the image size manageable (see Figure 2.2).
- **Symmetric Partitioning:** The dataset splits are balanced between normal and tumor samples. Following the common comparative protocol adopted in this project, the training set was limited to 80,000 balanced patches and the validation set to 8,000 patches. The independent test set contains 32,768 patches (16,391 healthy and 16,377 tumor-positive samples) and is used for the final evaluation of all architectures. While the classification reports use these exact test counts, the distribution plot in Figure 2.1 represents the ideal balanced split used to summarize the dataset.
- **Patient-Level Splitting:** The dataset partitions are separated at the patient level. This ensures that validation and test samples come from patients not seen during training, reducing data leakage and providing a more reliable estimate of model generalization.

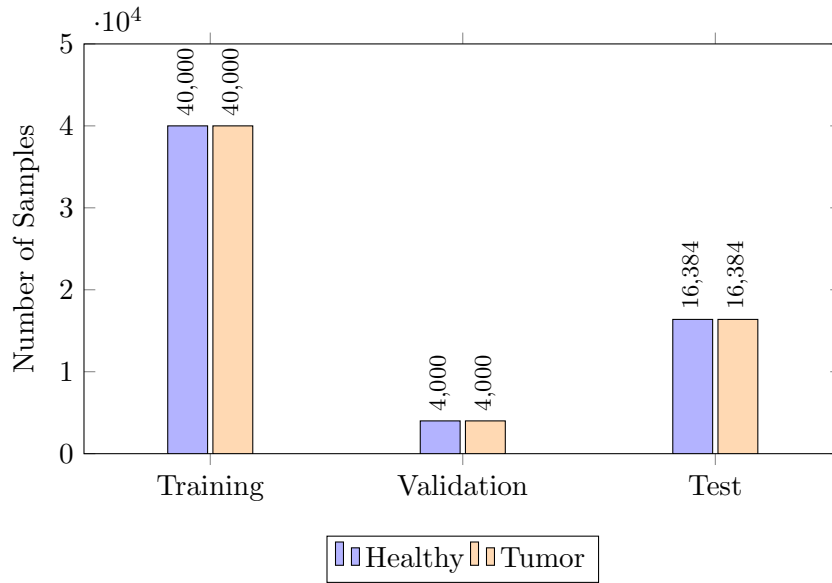


Figure 2.1: Distribution of working samples across the dataset splits. Each set maintains a strict 50/50 ratio to ensure unbiased learning.

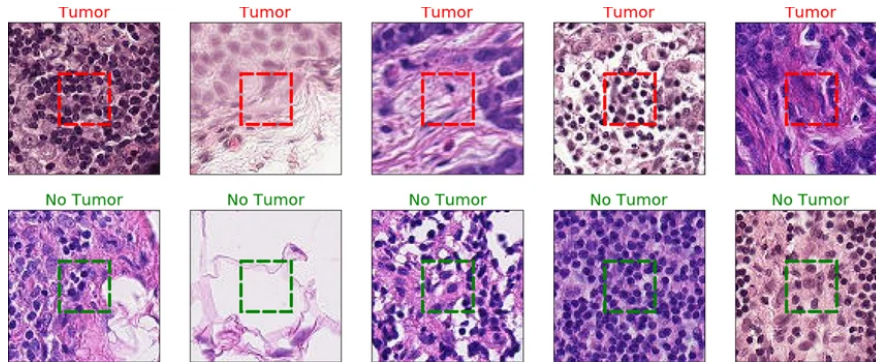


Figure 2.2: Some examples to compare healthy and diseased tissues.

2.2 Practical Implementation on Google Colab

Managing approximately 8GB of image data on Google Colab required an efficient data pipeline to reduce memory usage and avoid unnecessary GPU idle time:

- **Asynchronous Threading Pipelines:** We used the `tf.data` API to implement non-blocking data loading workflows. Data augmentation, including rotations, flips, and contrast shifts, together with pixel normalization, is executed through multithreaded CPU mapping (`num_parallel_calls=tf.data.AUTOTUNE`). In this way, upcoming batches are prepared asynchronously while the NVIDIA T4 GPU processes the current batch. Pixel intensities are scaled to the $[0, 1]$ range using division by 255.0 for the scratch-trained models, while the Transfer Learning pipeline uses the preprocessing function required by the pre-trained backbone.
- **HDF5 Out-of-Core Streaming:** Since loading the complete dataset into memory is not practical, the image arrays are stored and accessed through **HDF5 files**.

This layout allows the pipeline to stream data from storage during training, reducing RAM usage while keeping the batch processing workflow stable.

3. Methodology and Implementation

The methodology of this project is based on the comparison of different Deep Learning architectures for metastatic tissue classification. We follow an **”incremental complexity”** approach: we start from a custom convolutional baseline, then move to a residual architecture (ResNet), introduce geometric inductive bias through group-equivariant convolutions (G-CNN), and finally evaluate a Transfer Learning solution based on DenseNet121.

3.1 Experimental Design and Comparative Protocol

To make the comparison as fair as possible, we defined a common experimental protocol. The goal is to evaluate the impact of each architecture while keeping the main data and evaluation conditions consistent:

- **Data Standardization:** All models are trained and evaluated on the same patient-level data splits using a fixed random seed (42). The working dataset includes 80,000 training patches, 8,000 validation patches, and an independent test set of 32,768 patches. This allows us to compare the models under the same data conditions.
- **Loss Metric Optimization:** The custom architectures are trained with standard binary cross-entropy and symmetric class weights ($w_{healthy} = 1.0, w_{tumor} = 1.0$). This avoids introducing class-specific penalties during training and makes the comparison between scratch-trained models more direct.
- **Calibrated Decision Boundary:** Final test-set evaluations use a fixed classification threshold of 0.35 for all models. This value is lower than the default 0.50 because the task is considered as a screening problem, where missing a tumor-positive patch is more critical than producing an additional false positive. Using the same threshold for all models allows a direct comparison of their sensitivity to metastatic patterns.

3.2 Baseline Model: CNN "From-scratch"

The first architecture developed in this project is a custom Convolutional Neural Network (CNN) trained from scratch. This model is used as a baseline to understand how a relatively simple convolutional architecture performs before introducing deeper networks, geometric constraints, or pre-trained models. The architecture also includes regularization techniques to reduce overfitting and improve robustness to the noise and staining variability typical of Hematoxylin and Eosin (H&E) histopathological sections.

3.2.1 Data Streaming and Memory Caching Strategy

To reduce I/O overhead when working in Google Colab, the HDF5 files are copied into the local runtime storage (`/content/`). This avoids repeatedly accessing the dataset from slower remote-mounted storage during training.

Data loading is managed by the `load_to_ram` pipeline. For the baseline experiments, the function extracts 80,000 training samples and 8,000 validation samples. Since the PCam splits are already balanced, no additional online balancing procedure is required. This keeps the class distribution uniform from the beginning of training and avoids unnecessary preprocessing overhead.

3.2.2 Topological Architecture and Multi-Level Regularization

The custom baseline is composed of four convolutional stages that progressively increase the number of filters and learn increasingly abstract tissue features. The exact architecture is summarized in Figure 3.1:

1. **Controlled Input Spatialization:** Input images are cropped to a fixed size of $80 \times 80 \times 3$ pixels. This reduces part of the peripheral area while preserving enough context around the central 32×32 region that determines the ground-truth binary label.
2. **Symmetric Weight Regularization Framework:** All convolutional layers and the dense classification layer use **L2 weight regularization** with $\lambda = 5 \times 10^{-4}$. This penalizes large weights and helps reduce overfitting to local pixel-level patterns.
3. **Layer Hierarchy and Spatial Stabilization:** The model uses four convolutional blocks with 3×3 kernels and *ReLU* activations. The number of filters increases progressively as $[32 \rightarrow 64 \rightarrow 128 \rightarrow 256]$. Each convolutional output is normalized through a `BatchNormalization` layer and then downsampled using a 2×2 `MaxPooling2D` operation.
4. **Asynchronous Feature Map Deactivation:** The first three convolutional blocks include `SpatialDropout2D` layers with a dropout rate of 0.2. Unlike standard dropout, Spatial Dropout removes entire feature channels during training. This discourages the network from relying too much on specific color or texture maps and encourages more robust feature learning.
5. **Global Feature Aggregation:** Before the classification head, the final feature maps are reduced using `GlobalAveragePooling2D`. This avoids a large flattening layer, reduces the number of trainable parameters, and provides a compact representation for classification.

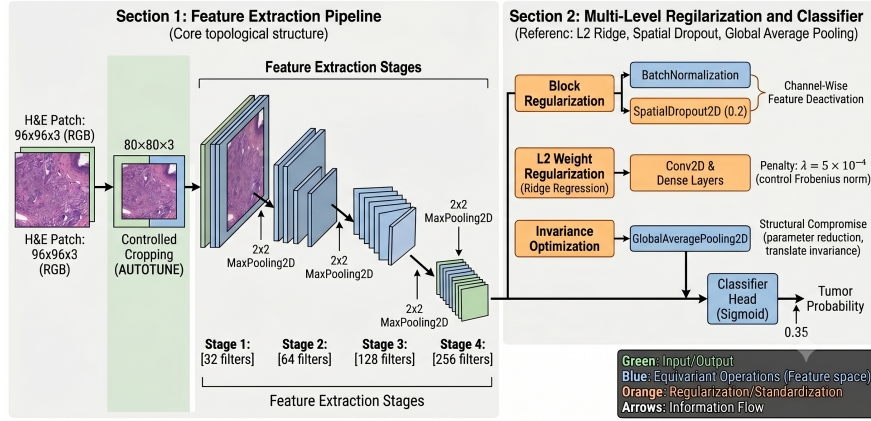


Figure 3.1: Topological architecture of the custom CNN From-Scratch baseline. The pipeline highlights the $80 \times 80 \times 3$ spatial resampling, the progressive 3×3 convolutional hierarchy ($32 \rightarrow 64 \rightarrow 128 \rightarrow 256$), and the multi-level regularization block integrating Spatial Dropout and L_2 weight penalty.

3.2.3 Training Execution and Accelerated Optimization Profile

The baseline classifier ends with a single dense neuron followed by a *Sigmoid* activation function, producing the probability of the input patch being tumor-positive. The classification head is kept in `float32`, while the hidden convolutional layers use a global **Mixed Precision policy** (`mixed_float16`) to improve training efficiency on the NVIDIA T4 GPU.

The model is trained using the Adam optimizer with an initial learning rate ($LR = 10^{-4}$) and a mini-batch size of 256 samples. Training is controlled through two call-backs:

- **Dynamic Learning Rate Decay:** `ReduceLROnPlateau` monitors the validation loss. If no improvement is observed for 3 epochs, the learning rate is reduced by a factor of $\gamma = 0.2$.
- **Early Termination Constraints:** `EarlyStopping` stops training if validation performance does not improve for 10 epochs and restores the best model weights observed during training.

Finally, the optimization uses **Just-In-Time (XLA) compilation** (`jit_compile=True`). This allows TensorFlow to optimize the computational graph and improve GPU execution efficiency.

3.3 Deep Residual Learning (ResNet)

After establishing the custom CNN baseline, the next step is to evaluate a deeper residual architecture. The goal is to increase model capacity while reducing the risk of vanishing gradients during backpropagation. For this reason, we implemented a custom architecture based on the principles of **Deep Residual Learning**.

3.3.1 Architectural Structure: The Residual Functional Mapping

The core component of this network is a custom `residual_block` function. Instead of forcing stacked layers to directly learn a target mapping $\mathcal{H}(x)$, the residual block learns a residual mapping $\mathcal{F}(x) := \mathcal{H}(x) - x$.

- **Identity Skip Connections:** Shortcut paths perform an element-wise addition (`layers.Add`) between the original input tensor x and the output of the internal convolutional transformations. This helps gradients flow more directly through the network during backpropagation.
- **Identity Projection and Dimension Matching:** Each block uses two consecutive 3×3 convolutions with L2 regularization ($\lambda = 5 \times 10^{-4}$). When the number of filters changes, the shortcut path applies a 1×1 convolution (`layers.Conv2D` with a 1×1 kernel) to match the tensor dimensions before the addition.
- **Post-Addition Activation:** After summing the residual path and the shortcut path, the result is passed through a `layers.Activation('relu')` layer. This introduces non-linearity after the residual merge.

Figure 3.2 shows the architecture of the ResNet.

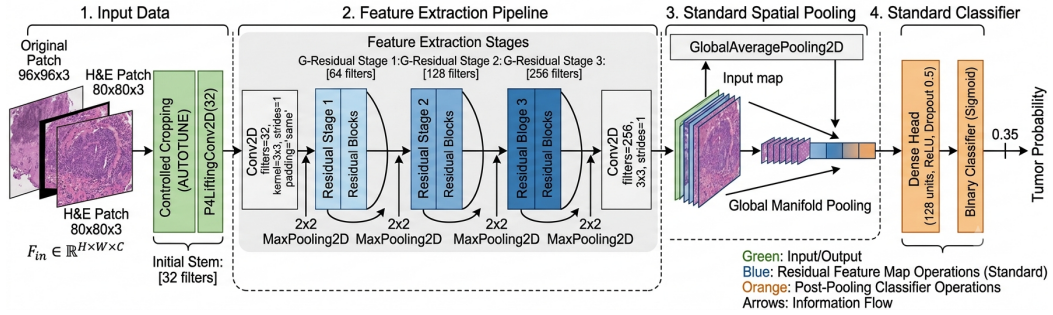


Figure 3.2: Topological scaling pipeline of a standard ResNet layout. Residual identity mappings bypass traditional stacked layers to directly preserve deep gradient propagation highways.

3.3.2 Data Management and Local Host Optimization

The ResNet model uses the same data loading strategy adopted for the baseline CNN:

- **Asynchronous Local Ingestion:** Consistent with the comparative protocol, the `load_to_ram` API loads a balanced subset of 80,000 training images and 8,000 validation images into the Colab runtime. This reduces I/O overhead during training.
- **Unified Spatial Boundaries:** Input images are processed to a fixed size of $80 \times 80 \times 3$ pixels. This keeps the input representation consistent with the baseline model and ensures that performance differences are mainly related to the residual architecture.

3.3.3 Topological Hierarchy and Optimization Schedule

The ResNet architecture combines the custom residual blocks into a progressive multi-scale feature extraction pipeline:

1. **Initial Feature Stem:** Before entering the residual stages, the $80 \times 80 \times 3$ inputs are processed by an initial 3×3 convolution with 32 filters followed by a `BatchNormalization` layer. This stage extracts low-level spatial features.
2. **Residual Processing Cascade:** The network then applies three residual stages with filter sizes $[64 \rightarrow 128 \rightarrow 256]$. Each residual block is followed by a 2×2 `MaxPooling2D` operation and a `SpatialDropout2D` layer with a dropout rate of 0.2.
3. **Classifier Head Configuration:** After the final 256-filter residual stage, the feature maps are reduced using `GlobalAveragePooling2D`. The resulting feature vector is passed to an intermediate dense layer of 128 units with *ReLU* activation, followed by a final sigmoid neuron compiled under global Mixed Precision rules (`mixed_float16`).
4. **Accelerated Optimization Infrastructure:** The model is trained with Adam ($LR = 10^{-4}$), a batch size of 256, and a maximum of 30 epochs. Training is controlled by `ReduceLROnPlateau`, which reduces the learning rate by a factor of 0.2 after validation stalls (patience = 3), and `EarlyStopping`, which stops training after 10 epochs without improvement and restores the best weights. The computational graph is compiled Just-In-Time via **XLA Optimization**(`jit_compile=True`) to improve GPU execution.

3.4 Hybrid G-CNN: Integrating Equivariance and Depth

The third architecture introduces geometric inductive bias through a Group Equivariant CNN. Standard convolutions are translation equivariant, but they do not explicitly handle rotations. This is a limitation for histopathological images, where tissue structures do not have a preferred orientation. The G-CNN addresses this by embedding rotational equivariance directly into the convolutional operations.

3.4.1 $p4$ Symmetry and Coordinate-Free Feature Representations

Histopathological tissue is approximately isotropic: cellular morphology and malignant nuclear patterns remain diagnostically relevant regardless of the orientation of the patch.

To exploit this property, the network introduces a geometric inductive bias through custom group-equivariant convolutional layers:

- **Mathematical Target Lifting:** The network replaces the first pixel-space convolution with a custom **P4LiftingConv2D** layer. In our implementation, this layer takes the standard three-channel input image and lifts its representation onto the $p4$ discrete rotation group. This is obtained by taking the base kernel tensor K and concatenating its rotated versions:

$$\mathcal{K} = \text{concat}([K, \text{rot}_{90}(K), \text{rot}_{180}(K), \text{rot}_{270}(K)])$$

along the filter axis using rotations implemented with `tf.image.rot90`. This allows the network to compute feature maps across the four orthogonal orientations.

- **Data-Driven Reflection Equivariance:** Rotation equivariance is handled by the custom convolutional layers, while reflection invariance is encouraged through data augmentation. Random flip operations are applied in the input pipeline using parallel CPU workers through `random_flip`, without adding extra trainable parameters.

3.4.2 Architectural Layout and Group Coordinate Collapsing

The global G-CNN architecture organizes the lifted group features into a multi-scale residual processing pipeline, as shown in Figure 3.3:

1. **Group-Equivariant Residual Pipeline:** The lifted feature maps pass through three sequential residual blocks (`g_residual_block`) with filter sizes $[64 \rightarrow 128 \rightarrow 256]$. Each convolutional block uses L2 regularization ($\lambda = 5 \times 10^{-4}$), while `SpatialDropout2D` layers (0.2) help reduce feature co-adaptation. Spatial downsampling is applied after each stage using standard 2×2 `MaxPooling2D` layers.
2. **Manifold Slicing and Group Pooling:** Before classification, the group-space tensors must be mapped back to a standard spatial representation. This is done by the custom **GroupPool** layer. The layer reshapes the channel dimension to separate the four rotation components and applies a pixel-wise maximum reduction:

$$\psi(x) = \max_{g \in p4} \mathcal{F}_g(x)$$

using `tf.reduce_max`. This keeps the strongest activation among the four rotations and produces a rotation-invariant representation.

3. **Classifier Execution:** The invariant feature maps are reduced using `GlobalAveragePooling2D` and passed to an intermediate dense layer of 128 units. The network ends with a single sigmoid classification neuron and uses Mixed Precision arithmetic (`mixed_float16`) together with Just-In-Time XLA graph compilation (`jit_compile=True`) to improve execution efficiency on the NVIDIA T4 GPU.

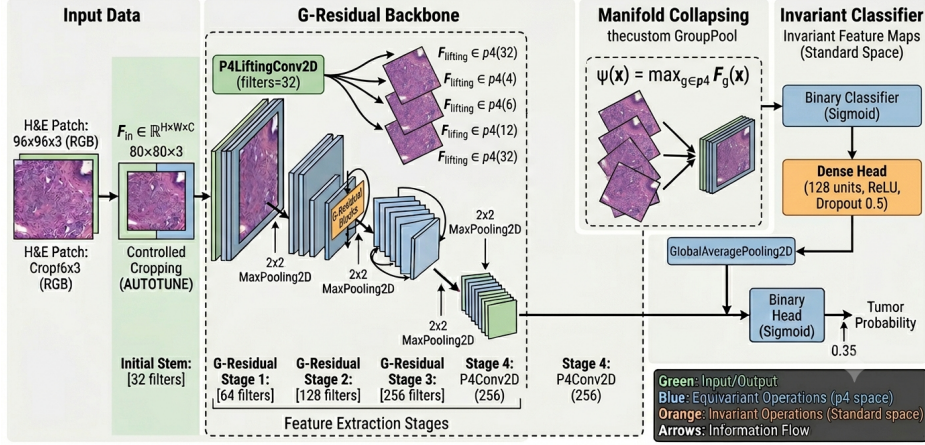


Figure 3.3: Architectural layout of the Group-Equivariant G-ResNet implemented in `GCNN.py`. The diagram illustrates the lifting convolution stage (*P4LiftingConv2D*) projecting input patches onto the discrete $p4$ symmetry group, the equivariant residual backbone, and the custom *GroupPool* layer executing manifold collapsing ($\max_{g \in p4}$) for rotation-invariant tissue classification.

3.5 Transfer Learning: DenseNet121 Suite

The final model uses **DenseNet121** as a pre-trained feature extractor. DenseNet architectures are based on dense connections, where each layer receives input from all previous layers within the same dense block. This improves feature reuse and helps preserve useful low-level visual information across deeper layers.

3.5.1 Preprocessing Customization and Data Augmentation Architecture

Our implementation adapts the ImageNet-pretrained backbone to the histopathological classification task. Input images are kept at $96 \times 96 \times 3$ pixels to match the expected input size of the DenseNet backbone. The data workflow includes the following processing steps:

- **Color and Staining Robustness Optimization:** Histopathological images may vary due to staining and illumination differences. To reduce sensitivity to these variations, the training pipeline applies random contrast adjustments in the range $[0.75, 1.25]$ and random brightness changes of 0.15. This encourages the model to focus more on tissue morphology than on color intensity alone.
- **Pre-trained Normalization Alignment:** Before being passed to DenseNet121, images are processed using the official `applications.densenet.preprocess_input` function. This maps the input distribution to the format expected by the ImageNet-pretrained backbone.

3.5.2 Two-Stage Asymmetric Optimization Scheme

To adapt the model to the screening objective, the DenseNet121 backbone is followed by a custom classification head composed of a dense layer with 256 units, *ReLU* activation, *BatchNormalization*, and *Dropout* (0.5).

Unlike the scratch-trained models, DenseNet121 is trained with an asymmetric class-weighting strategy ($w_{\text{tumor}} = 2.5, w_{\text{healthy}} = 1.0$), motivated by the clinical priority of reducing missed tumor-positive samples:

1. **Partial Backbone Feature Freezing (Warm-up Phase):** During the first 3 epochs, the classification head is initialized while most of the backbone remains frozen. Inside the `build_transfer_model` function, the earlier layers are locked and only the last part of the backbone is allowed to adapt (`layers[: -30]`). The model is compiled with a learning rate of $1e - 3$ during this phase.
2. **Global Relaxation and Label-Smoothed Fine-Tuning:** In the second phase, the entire DenseNet121 backbone is unfrozen (`base_model.trainable = True`) and fine-tuned for 25 epochs. A smaller learning rate ($1e - 5$) is used to adapt the pre-trained features without destabilizing them. The objective function also includes **Label Smoothing** (0.1), which reduces overconfident predictions and stabilizes training under asymmetric class weights.

3.5.3 Out-of-Core Test Ingestion Pipeline

A practical implementation challenge is evaluating the full test set of 32,768 patches at a 96×96 resolution without overloading RAM. Loading all images at once would increase the risk of memory issues in the Colab environment.

To address this, validation and test inference use an out-of-core streaming pipeline based on `tf.data.Dataset.from_generator`. The test images remain stored in the local HDF5 files and are loaded sequentially through a generator. The resulting batches are processed asynchronously (256 samples) and prefetched using `tf.data.AUTOTUNE`. This allows the model to run inference on the full test set while keeping memory usage under control.

In Figure 3.4 the topological view of the architecture of DenseNet model.

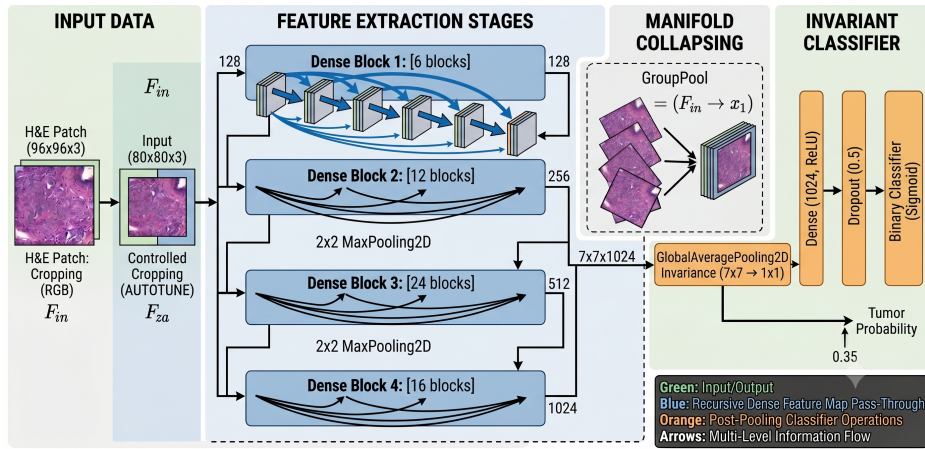


Figure 3.4: Structural connectivity matrix inside a baseline Dense Block setup. Each internal processing layer receives unified direct inputs from all historical preceding blocks, maximizing out-of-domain feature reuse configurations.

4. Results and Discussion

In this chapter, we evaluate the performance of the developed neural architectures on the independent patient-level test set, composed of 32,768 patches. The analysis follows three steps: first, we examine the training dynamics through learning curves; second, we compare the models using global quantitative metrics and ROC-AUC analysis; finally, we analyze the confusion matrices to better understand the clinical impact of the classification errors.

4.1 Training Dynamics: Learning Curves

Before discussing the final classification results, it is useful to observe the training behavior of each model. Figure 4.1 shows the Loss and Accuracy curves monitored on both the training and validation sets.

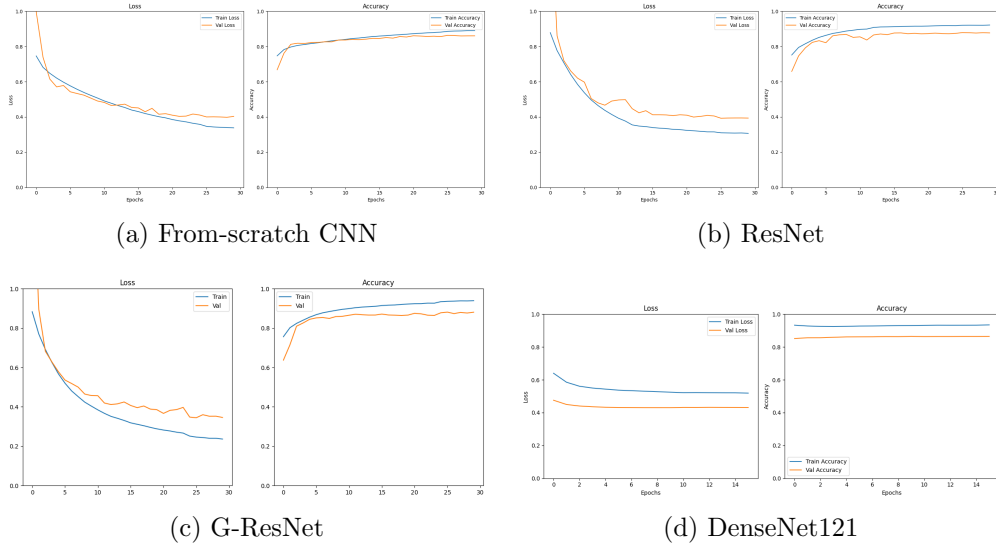


Figure 4.1: Learning curves (Loss and Accuracy) across all architectures. The scratch-trained configurations require more epochs to stabilize, while DenseNet121 benefits from pre-trained weights and a conservative learning rate ($1e-5$).

The training curves show different convergence behaviors across the evaluated models:

- **From-scratch CNN (a):** This model acts as the baseline. It shows stable training, but it reaches a lower accuracy plateau compared to the more advanced architectures. This suggests that the model has limited capacity to capture more complex morphological features of metastatic tissue.

- **ResNet-50 (b):** The ResNet model reaches higher training accuracy, but it also shows a visible generalization gap and validation loss instability between epochs 5 and 15. This suggests that training a deeper residual architecture from scratch is more difficult in this setting. This behavior is consistent with the lower Tumor Recall (0.65) observed in the final evaluation.
- **G-ResNet (c):** The G-ResNet shows a smoother validation behavior compared to the standard ResNet. The use of group-equivariant convolutions appears to help the model handle rotational variability in the tissue patches, improving robustness without relying only on increased depth.
- **DenseNet121 (d):** The Transfer Learning pipeline reaches high accuracy early in training. The validation curves are more stable, showing the advantage of starting from pre-trained features and fine-tuning them on the PCam classification task.

Overall, the learning curves suggest that models trained from scratch are more sensitive to optimization instability, especially when the architecture becomes deeper. DenseNet121, instead, benefits from Transfer Learning and provides a more stable training process. This makes it a strong candidate for the screening-oriented objective of the project.

4.2 Global Performance Comparison

Table 4.1 reports the quantitative evaluation metrics computed on the test set. All models are evaluated using the same decision threshold of 0.35, in order to compare their behavior under the same screening-oriented condition.

Model	Accuracy	Macro F1-Score	Recall		Precision		AUC
			Tumor	Healthy	Tumor	Healthy	
From-Scratch	0.75	0.75	0.71	0.85	0.78	0.80	0.91
G-CNN	0.83	0.83	0.77	0.88	0.88	0.79	0.92
ResNet	0.75	0.75	0.65	0.86	0.82	0.71	0.93
DenseNet121	0.84	0.84	0.85	0.83	0.83	0.84	0.91

Table 4.1: Global performance comparison on the PCam test set under the unified screening protocol ($Threshold = 0.35$). Bold values indicate the top performer for each metric.

4.2.1 Discriminative Capacity: ROC and AUC Analysis

Accuracy, Precision, and Recall depend on the selected decision threshold. For this reason, we also analyze the ROC curve and the corresponding Area Under the Curve (AUC), which provide a threshold-independent view of the discriminative ability of each model.

Figure 4.2 shows the ROC curves computed on the test set. The **From-Scratch CNN** obtains an AUC of 0.91, representing a solid baseline. The **ResNet-50** reaches the highest AUC value (0.93), showing that the model is able to separate the two classes well when considered independently from a fixed threshold. However, this does not directly translate into the best Tumor Recall at the selected operating point.

The **Hybrid G-ResNet (G-CNN)** obtains an AUC of 0.92, confirming that the geometric inductive bias introduced through the $p4$ symmetry group is useful for this task. DenseNet121 reaches an AUC of 0.91, which is slightly lower than ResNet and G-CNN, but it provides the best screening-oriented balance because it achieves the highest Tumor Recall.

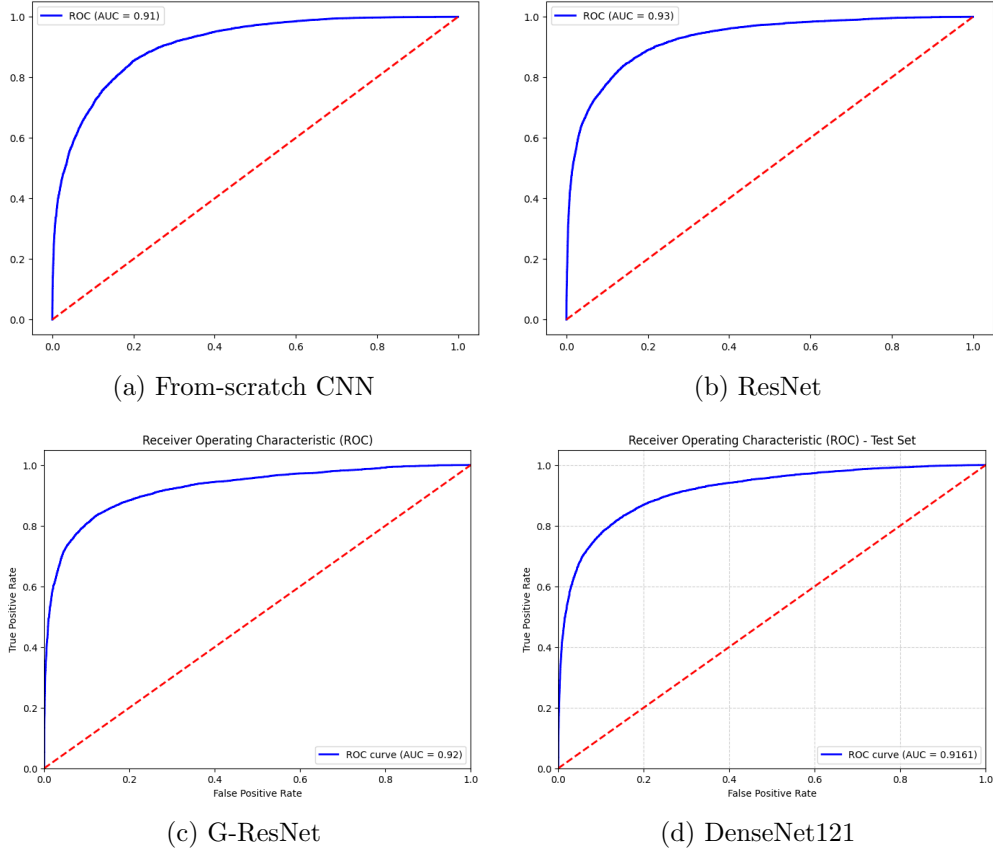


Figure 4.2: ROC curves and absolute AUC values computed over the test data distribution.

4.2.2 Architectural and Clinical Discussion

The quantitative comparison under the unified threshold of 0.35 highlights how the different architectures behave in a screening-oriented setting.

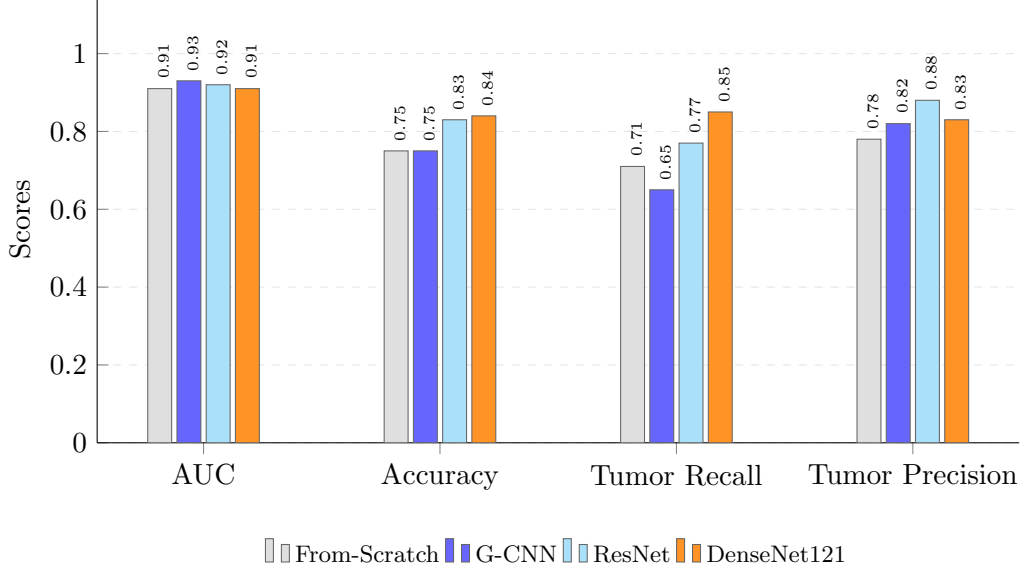


Figure 4.3: Performance comparison across the evaluated architectural paradigms on the PCam test benchmark.

The From-Scratch CNN: A Baseline for Sensitivity

The **From-Scratch** CNN provides a reasonable baseline, with an Accuracy of 0.75 and an AUC of 0.91. At the selected threshold, it obtains a Tumor Precision of 0.78 and a Tumor Recall of 0.71. These results show that the regularized CNN can learn useful tissue features, but its limited depth reduces its ability to capture more complex malignant patterns.

ResNet-50: The Optimization Bottleneck

The **ResNet-50** architecture obtains the highest AUC value (0.93), showing strong threshold-independent discriminative ability. However, under the fixed threshold of 0.35, it reaches the lowest Tumor Recall (0.65). This means that, despite its overall separation capability, the model misses more tumor-positive samples than the other architectures. The learning curves suggest that this behavior is related to optimization instability and overfitting when training a deeper residual model from scratch.

Hybrid G-ResNet: Geometric Intelligence

The **Hybrid G-ResNet (G-CNN)** improves over the standard ResNet in the main screening metrics. It reaches an Accuracy of 0.83, an AUC of 0.92, a Tumor Recall of 0.77, and the highest Tumor Precision (0.88). This suggests that incorporating rotational equivariance through the p_4 group helps the model learn more stable representations of histopathological images, where tissue orientation is not diagnostically relevant.

DenseNet121: The Clinical Peak

The **DenseNet121** Transfer Learning model provides the best screening-oriented trade-off in our experiments. Thanks to pre-trained ImageNet features, data augmentation, and asymmetric class weighting ($w_{tumor} = 2.5$), the model overcomes part of the sensitivity limitations observed in the scratch-trained architectures.

At the threshold of 0.35, DenseNet121 achieves the highest Accuracy (0.84), the highest Macro F1-Score (0.84), and the highest Tumor Recall (0.85), while maintaining a balanced Tumor Precision of 0.83. Although its AUC is not the highest, this model is the most aligned with the clinical objective of reducing False Negatives.

4.3 Qualitative Analysis: Confusion Matrices

The confusion matrices in Figure 4.4 provide a more direct view of the classification errors. In particular, they show how each model distributes errors between False Positives and False Negatives.

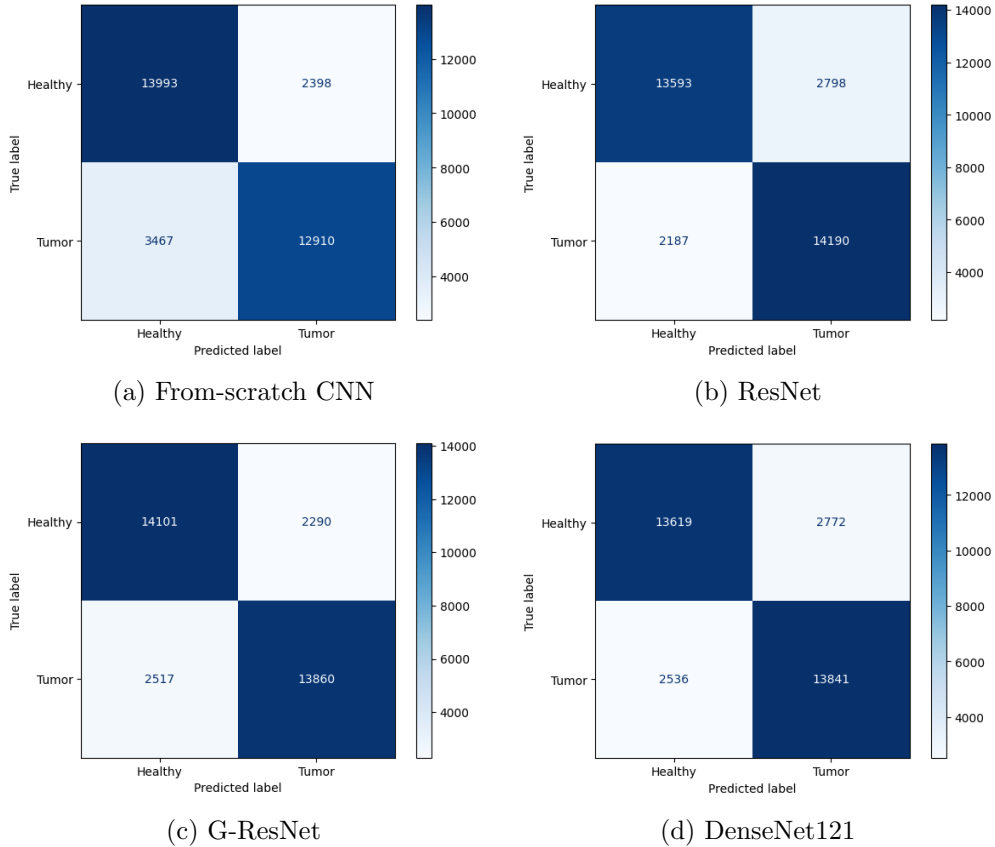


Figure 4.4: Confusion Matrix comparison evaluated across the test data samples.

4.4 State of the Art

To contextualize the performance of our models, we refer to the foundational work by *Veeling et al. (2018)* [veeling2018rotation](#), who introduced the PCam benchmark and provided an important reference point for this dataset.

Geometric Inductive Biases Veeling et al. showed that histopathological images can benefit from models that account for rotational and reflectional symmetries. Our **Hybrid G-ResNet (G-CNN)** experiment follows this idea by using convolutional kernels constrained within the $p4$ symmetry group. The results support this direction: compared to the standard ResNet, the G-CNN achieves higher Accuracy, higher Tumor Recall, and higher Tumor Precision. This suggests that architectural inductive biases can be useful when working with tissue images, where orientation should not affect the diagnostic content.

Transfer Learning and Resource Constraints A difference between our work and the original benchmark methodology is the use of **Transfer Learning**. While Veeling et al. 2018 used custom architectures trained from scratch, our project was developed in a resource-constrained environment using Google Colab. In this setting, training deeper architectures such as ResNet-50 from scratch was less effective, as shown by the generalization gap and lower Tumor Recall.

By using the **DenseNet121** backbone pre-trained on ImageNet, we were able to obtain competitive results with lower computational cost. The use of a manually selected threshold (0.35) and asymmetric class weighting ($w_{tumor} = 2.5$) also helped prioritize the clinical objective of reducing missed tumor-positive samples.

Global Benchmarking Current state-of-the-art results on the PCam dataset usually reach AUC values between 0.96 and 0.98. However, these results often rely on larger models, ensembles, Vision Transformers, or Self-Supervised Learning methods trained on much larger collections of histopathological images.

- **Academic SOTA:** Typically focuses on maximizing benchmark performance, often using more complex models or larger training setups.
- **Our Approach:** Focuses on a computationally feasible single-model pipeline. In this setting, achieving an **AUC of 0.91** and a **Tumor Recall of 0.85** with DenseNet121 represents a reasonable result under consumer-grade hardware constraints.

Overall, our findings suggest that combining geometric inductive bias, Transfer Learning, and cost-sensitive tuning can be an effective direction for computer-aided metastatic tissue classification.

References

- Ai usage (2026). *Artificial intelligence tools were used to support brainstorming and code development during the preparation of this report.*
- Bejnordi, Babak Ehteshami et al. (2017). “Diagnostic Assessment of Deep Learning Algorithms for Detection of Lymph Node Metastases in Women With Breast Cancer”. In: *JAMA* 318.22, pp. 2199–2210.
- Veeling, Bastiaan S. (2018). *PatchCamelyon (PCam) Dataset*. GitHub Repository. URL: <https://github.com/basveeling/pcam> (visited on 06/08/2024).
- Veeling, Bastiaan S. et al. (2018). “Rotation equivariant CNNs for digital pathology”. In: *Medical Imaging with Deep Learning (MIDL)*. URL: <https://arxiv.org/abs/1806.03962>.